binary search in aray python

Here's a simple implementation of **Binary Search** in Python:

Binary Search (Iterative)

python Copy Edit

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid  # Target found, return index
        elif arr[mid] < target:
            left = mid + 1  # Search in the right half
        else:
            right = mid - 1  # Search in the left half

    return -1  # Target not found

# Example usage
arr = [2, 3, 4, 10, 18, 20, 25]
target = 10
result = binary_search(arr, target)

if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")
```

Binary Search (Recursive)

python Copy Edit

```python
def binary_search_recursive(arr, left, right, target):
    if left > right:
        return -1  # Target not found

    mid = left + (right - left) // 2

    if arr[mid] == target:
        return mid  # Target found
    elif arr[mid] < target:
        return binary_search_recursive(arr, mid + 1, right, target)  # Search right
    else:
        return binary_search_recursive(arr, left, mid - 1, target)  # Search left

# Example usage
arr = [1, 5, 8, 12, 20, 33, 45]
target = 20
result = binary_search_recursive(arr, 0, len(arr) - 1, target)

if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")
```

🔹 **Time Complexity**:

- Best Case: **O(1)** (if the middle element is the target)
- Worst/Average Case: **O(log n)** (dividing the array in half each time)

Make sure the array is **sorted** before applying Binary Search! 🚀

how its works, explaain

How Binary Search Works?

Binary Search is an efficient **search algorithm** that works on a **sorted array** by repeatedly dividing the search range in half. It follows the **divide and conquer** approach.

Step-by-Step Explanation

Let's assume we have a sorted array:

```python
arr = [2, 3, 4, 10, 18, 20, 25]
target = 10
```

We want to find **10** in this array.

Step 1: Find the Middle Element

1. Calculate the **middle index**:

$$\text{mid} = \frac{\text{left} + \text{right}}{2}$$

Initially, `left = 0`, `right = 6` (length of array - 1).

$$\text{mid} = \frac{0 + 6}{2} = 3$$

The middle element is **10**.

2. **Check if mid == target**
   - **Yes**, we found **10** at index **3** → **Search ends** ✅

Example with a Different Target

Let's say we want to find `18` instead.

Step 1: Find the Middle Element

- `left = 0`, `right = 6`
- `mid = (0 + 6) // 2 = 3`
- `arr[mid] = 10` (not 18)

Step 2: Decide the Search Direction

- Since `10 < 18`, we **ignore the left half** and search in the **right half**.
- New `left = mid + 1 = 4`, `right = 6`.

Step 3: Find New Middle Element

- `mid = (4 + 6) // 2 = 5`
- `arr[mid] = 20` (greater than 18)

Step 4: Search Left Again

- Since `20 > 18`, search the **left half** → `left = 4`, `right = mid - 1 = 4`.

Step 5: Find New Middle

- `mid = (4 + 4) // 2 = 4`
- `arr[mid] = 18` 🎯 **Found!**

Time Complexity

- **Best Case:** `O(1)` (if the middle element is the target)
- **Worst/Average Case:** `O(log n)` (dividing array in half each time)

🔹 **Example of Division:** If we have **n = 8** elements:

- 1st check → 4 elements left
- 2nd check → 2 elements left

- 3rd check → 1 element left
  Thus, **$\log_2(n) \approx 3$** steps for `n = 8`.

Key Points

- ✅ Works only on a **sorted array**
- ✅ Reduces search space by **half** each step
- ✅ Much faster than **linear search** (`O(n)`)

Let me know if you need further clarifications! 🚀

New left = mid + 1 = 4, right = 6. : explain this

Why does `left = mid + 1` and `right = 6`?

Let's break it down step by step.

Recap: Our Array and Target

We are searching for **18** in the sorted array:

```python
arr = [2, 3, 4, 10, 18, 20, 25]
target = 18
```

Initially:

- `left = 0` (first index)
- `right = 6` (last index)

Step 1: Find the First Middle Element

- Compute `mid = (left + right) // 2 = (0 + 6) // 2 = 3`
- `arr[mid] = 10`

Since `10` is **less than** `18`, this means:

1. Our target **must be on the right side** of the array.
2. The left half **[2, 3, 4, 10]** can be ignored.

So, we move `left` to `mid + 1`:

```python
left = mid + 1 = 3 + 1 = 4
```

The `right` remains the same (`right = 6`).

Why Ignore the Left Half?

We know the array is **sorted**. Since `10 < 18`, all elements **before index 3** are also smaller than `18`.
Thus, there's **no need to check them again**, and we move `left` forward.

Now, the new **search range** is:

```python
arr = [18, 20, 25]  # We only search within this part
   ↑    ↑    ↑
 left(4) mid(?) right(6)
```

Step 2: Find the New Middle Element

- `mid = (4 + 6) // 2 = 5`
- `arr[mid] = 20`

Since `20` is **greater than** `18`, this means:

1. Our target **must be on the left side** of `mid`.
2. The right half **[20, 25]** can be ignored.

So, we move `right` to `mid - 1`:

python Copy Edit

```
right = mid - 1 = 5 - 1 = 4
```

Step 3: Find the New Middle Element

Now, the updated search range is:

python Copy Edit

```
arr = [18]  # Only one element left
   ↑
 left(4), right(4)
```

- `mid = (4 + 4) // 2 = 4`
- `arr[mid] = 18` ✅ **Found!**

Key Concept:

- If `arr[mid] < target`: Move `left = mid + 1` (search in the right half)
- If `arr[mid] > target`: Move `right = mid - 1` (search in the left half)
- If `arr[mid] == target`: **Element found!**

Let me know if anything needs more clarification! 🚀

now give a simple pythn code for it

Here's a **simple Python code** for **Binary Search**:

python Copy Edit

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2  # Find the middle index

        if arr[mid] == target:
            return mid  # Target found, return index
        elif arr[mid] < target:
            left = mid + 1  # Search in the right half
        else:
            right = mid - 1  # Search in the left half

    return -1  # Target not found

# Example usage
arr = [2, 3, 4, 10, 18, 20, 25]  # Sorted array
target = 18  # Element to search

result = binary_search(arr, target)
```

```
if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")
```

How it Works?

1. **Find the middle element**.
2. **If it's the target, return the index**.
3. **If target is greater, search the right half** (`left = mid + 1`).
4. **If target is smaller, search the left half** (`right = mid - 1`).
5. **Repeat until found or** `left > right` **(element not in the array).**

🚀 **Time Complexity:** `O(log n)`

Let me know if you need any modifications! 😊

```
from array import *
array1 = array('i', [10, 23,35,56,82,100])
n = int(input("Enter the number to search: "))
def binary_search():
   while(min<max):
      mid = (min+max)//2
      if(array1[mid]==n):
         return mid
      elif(array1[mid]<n):
         min = mid+1
      else:
         max = mid-1
```

Your code has a few issues that need to be corrected for it to work properly. Here's the **corrected version** with explanations:

Issues in Your Code

1. `min` **and** `max` **are not defined before use**
   - You need to initialize them at the start of the function.
2. **Function should accept parameters**
   - Instead of accessing global variables (`array1` and `n`), pass them as parameters.
3. **Incorrect while loop condition**
   - `while(min < max)` should be `while min <= max`, otherwise, it might miss checking the last element.
4. **Modifying** `min` **and** `max` **inside the function without** `global`
   - Since `min` and `max` are local to the function, Python will throw an **UnboundLocalError** when you try to modify them.

Fixed Code

python Copy Edit

```
from array import *

def binary_search(arr, n):
    left, right = 0, len(arr) - 1  # Initialize boundaries

    while left <= right:
        mid = (left + right) // 2  # Find middle index

        if arr[mid] == n:
            return mid  # Element found, return index
        elif arr[mid] < n:
            left = mid + 1  # Search in the right half
        else:
            right = mid - 1  # Search in the left half
```

```
    return -1  # Element not found

# Array declaration
array1 = array('i', [10, 23, 35, 56, 82, 100])

# User input
n = int(input("Enter the number to search: "))

# Function call
result = binary_search(array1, n)

# Output result
if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")
```

Corrections & Improvements

- ✅ **Pass array and target number as parameters** (no global variables)
- ✅ **Define `left` and `right` properly**
- ✅ **Fix `while min < max` → `while left <= right`**
- ✅ **Fix variable naming (`min` and `max` are reserved keywords)**

Let me know if you have any questions! 🚀😊

```
from array import *
array1 = array('i', [10, 23,35,56,82,100])
n = int(input("Enter the number to search: "))
def binary_search():
   while(min<max):
      mid = (min+max)//2
      if(array1[mid]==n):
         return mid
      elif(array1[mid]<n):
         min = mid+1
      else:
         max = mid-1 : correct it
```

There are several issues in your code that need to be corrected:

Errors in Your Code

1. **`min` and `max` are not defined before use**
   - You need to initialize them before the `while` loop.
2. **Incorrect while loop condition**
   - `while(min < max)` should be `while min <= max`, otherwise, the last element might not be checked.
3. **Using `min` and `max` inside the function without defining them**
   - `min` and `max` need to be initialized as `left` and `right` inside the function.
4. **Using `min` and `max` as variable names**
   - `min` and `max` are built-in Python functions, so it's better to use `left` and `right` instead.
5. **Modifying `min` and `max` without `global` keyword**
   - This would cause a **UnboundLocalError**.

Fixed Code

python Copy Edit

```
from array import *

def binary_search(arr, n):
```

```
    left, right = 0, len(arr) - 1  # Initialize search range

    while left <= right:
        mid = (left + right) // 2  # Find middle index

        if arr[mid] == n:
            return mid  # Target found, return index
        elif arr[mid] < n:
            left = mid + 1  # Search in the right half
        else:
            right = mid - 1  # Search in the left half

    return -1  # Target not found

# Array declaration
array1 = array('i', [10, 23, 35, 56, 82, 100])

# User input
n = int(input("Enter the number to search: "))

# Function call
result = binary_search(array1, n)

# Output result
if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")
```

Corrections & Improvements

✅ **Replaced** `min` **and** `max` **with** `left` **and** `right` (avoids overriding built-in functions)
✅ **Defined** `left` **and** `right` **before the while loop**
✅ **Fixed** `while(min < max)` → `while left <= right`
✅ **Properly modifying** `left` **and** `right` **inside the function**

Now, this code will correctly perform **binary search** on the array! 🚀😊

give a promt for cursor ai to apply light theme to the sections shown in image , with proper functionality

Here's a **Cursor AI prompt** to apply a **light theme** to the sections shown in the image, ensuring proper functionality:

Prompt:
*"Modify the UI of the weather forecast application to apply a **light theme** with the following specifications:

1. **Header Styling**
   - Change the background to a soft white or light pastel gradient.
   - Ensure the 'Weather ForeCasts' title has a vibrant gradient text (pink & purple).
   - Keep the subtitle ('Your Daily Weather Companion') in a **dark gray or soft black** for readability.
2. **Main Section (Location & Temperature)**
   - Use a **subtle light background** with soft drop shadows for depth.
   - 'Ashta' (city name) should be in bold purple, and 'India' in **dark gray**.
3. **Information Boxes (Max Temp, Min Temp, Time, Location)**
   - Apply a **light gray or white background** with rounded corners.
   - Change the text color to a dark shade for readability.

- Ensure the **icons remain visible and match the theme**.

4. **Top Right Controls (Theme Toggle & Update Time)**
   - Ensure the **dark mode toggle works smoothly**, switching themes dynamically.
   - The 'Updated Time' text should be in dark gray with a light button background.
5. **Functionality**
   - Maintain full **responsiveness and animations**.
   - Ensure smooth transitions between dark and light modes.
   - Validate proper alignment and spacing after applying the light theme."*

This will guide **Cursor AI** to **apply the light theme** while maintaining readability and functionality. 🚀
Let me know if you need refinements! 😊